

RESEARCH

Open Access



A systematic evaluation of LSTM, XGBOOST and hybrid models for intraday trading using a risk-focused machine learning framework

Lokesh Kumar Soni^{1,2}, Mangal Singh¹, Aditya Jain¹, Pranita Soni^{1,3}, Amit Kumar Goyal^{4*} and Ajay Kumar⁵

*Correspondence:

Amit Kumar Goyal

amit.goyal@manipal.edu

¹Symbiosis Institute of Technology,
Pune Campus, Symbiosis
International (Deemed University),
Pune, Maharashtra 412115, India

²Bharat Electronics Limited,
Pune 411021, India

³Dr. D.Y. Patil Institute of
Technology, Pimpri, Pune
411018, India

⁴Manipal Institute of Technology,
Manipal Academy of Higher
Education, Manipal 576104, India

⁵Department of Electronics and
Communication Engineering,
Jaypee Institute of Information
Technology, 201309, Noida, India

Abstract

The stock market is an important component of personal finance and presents many investment instruments, including equities, mutual funds, and trading. Growing awareness and the availability of technology have led more people to start trading with the view to realizing significant financial gains and security over the long run. In the era of advanced technologies, the financial market is no exception to their influence. The main goal of the study is to disprove the popular myth that high predictive accuracy is the only reason behind high returns. Rather, it highlights how the structure and execution of an effective trading infrastructure is more important in delivery of a stable financial performance. The purpose of this paper is to develop a systematic improvement and comparative analysis which would help to evolve a simple predictive model with a standard accuracy of 0.53 into a strong framework with the ability to attain 10X financial goals. The performance gains seen can be attributed to the systematic combination of quantitative risk management techniques, which include algorithmically specified stop-loss limits and position sizing rules, and the compounding effect that causes the capital to grow exponentially within the trading structure. The scientific contribution of this study lies in bridging predictive modeling and trading system design, demonstrating that the integration of risk management, transaction costs, and compounding dynamics plays a decisive role in realized trading performance, beyond predictive accuracy alone. The experimental framework setup consists of four selected models: LSTM, XGBOOST, an integrated XGBOOST–LSTM model, and XGBOOST augmented with moving average features.

1 Introduction

1.1 Background

Over the past few years, machine learning has slowly researched the formulation and utilization of real-time stock forecasting systems, particularly as the markets have been increasingly responsive to transient indicators and non-linear variations. Although rule-based strategies have a role in certain stable or characteristically constrained environment, it does not work well when confronting the high-speed and uncertainty



© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

of intraday trading dynamics. In that sense, recurrent neural networks and gradient-boosted decision trees models have been found to be more sensitive, especially when they are trained over multiple time horizons in order to capture short-term momentum and the overall trend changes [1]. What is interesting is that these models become even more useful once they are implemented in a trading system that does take into consideration position sizing and risk limits-both of which can be borrowed by the traditional portfolio theory. As an example, consider times of increased volatility, such as in spikes in the VIX, where rigid systems tend to be oblivious to the change in market sentiment, adaptive models will possibly reduce position sizes depending on the confidence in the prediction. Naturally, no model can be lossless, however, adding the aspect of behavioral finance, like, changes in overreaction or the tendency to mean revert, can add this down to earth nearly hunch back to the decision-making process of a model. This combination of predicted algorithms and practical trading reasoning is not a matter of perfection, it is a matter of approximating the way seasoned market participants think and behave in the real world.

Back-testing is an integral component of designing any form of data-driven trading model and provides the retroactive perspective from which the predictive capability and usability of a model can be assessed [2]. By simulating trading activities on past data, the researchers are able to assess the basic predictive accuracy, as well as the economic significance of the predictions, when translated into trading signals. This is done in such a way that it involves a critical component such as the careful division of the dataset into training, validation, and testing subsets. The learning of the model is facilitated by the training set, the hyperparameter search and prevention of overfitting by the validation set, and an objective assessment of the performance in the real world (without developing the model) by the testing set. This step-by-step procedure allows the comparison of the merits of some different candidate models such as an LSTM-based model, XGBOOST's regression models, or hybrid models, which are all assessed on the basis of market returns, besides other metrics such as mean squared error, directional accuracy, and risk-adjusted returns.

The existing literature has improved various machine learning models in the prediction of stock market movement [3]. Two of the most popular that have been used most recently are Extreme Gradient Boosting (XGBOOST) and Long Short-Term Memory (LSTM). As an extension of RNN, LSTM is well adapted to work with sequential data and has proven highly effective in capturing long-term dependency, making it be in particular useful to analyze the market trends. XGBOOST on the other, is a highly efficient gradient boosting algorithm with structured data sets that are known to be high performance and accuracy-wise. This hence makes it very suitable to financial time-series data set as it enhances the predictive power and nonlinear relationship in the data can be discovered. In order to assess the complexity of market trends, the present study devises a novel hybrid model in balancing XGBOOST and LSTM (XGBOOST + LSTM) and, therefore, leveraging the benefits of each other to provide the capabilities of predictive accuracy. The suggested outcome hybrid model of XGBOOST with Moving Averages (XGBOOST + MA) would be a combination of the sophisticated forecasting capability of XGBOOST and technical analysis indicators tested by the market with an emphasis on accuracy improvement.

The technical analysis is all about moving averages, which are employed to eliminate the volatility of the price in the short term and indicate the general market trends. Along with XGBOOST, these indicators are a part of a more flexible and data-driven trading system—the one that would identify the nonlinear trends without sacrificing the feel of the traditional system. The XGBOOST + MA framework is especially appealing from the perspective of speed and efficiency. Compared to the LSTM models, which are extremely effective in modeling and training time-based sequences but have more complex computational requirements linked to them, XGBOOST provides faster predictions, which is a big advantage in real-time trading systems. The latency advantage could be the difference between having a competitive advantage or losing a competitive advantage in markets where milliseconds matter. And the current study assesses the performance of selected LSTM and XGBOOST models instead of discussing all recent architectures. Future works can also extend the analysis to linear baselines, other boosting techniques, and transformer-based models, as well as more realistic consideration in intraday trading.

1.2 Literature review

Sentiment is not a nicety in the contemporary markets but a requirement [4]. The price action is normally driven by perception as much as fundamentals with the emergence of social media and 24/7 financial news cycles. Through the tone of the financial reporting and monitoring the language patterns of the platforms, such as Twitter or Reddit, the traders have a distinct advantage, particularly in the volatile swings [5]. Superimposed on the traditional indicators, the sentiment analysis offers information in the interpretation of the market—the existence of forces in the market that are emotional in nature. The flexibility is what is particularly powerful about the contemporary machine learning algorithms. These models possess the capability to adjust to the market, learn from the new information, and adjust strategies on the fly. Even more, the ensemble methods, where multiple models are combined, have proved particularly useful. With the amalgamation of the strengths of multiple algorithms, they are likely to offer not only a higher level of accuracy but also higher consistency, particularly in scenarios where the markets are unpredictable [6]. However, relative to the traditional methods such as CNN, RNN, and LSTM, the Transformer deep learning model [7] is more accurate.

Decision trees (DTs) are a prevalent technique within machine learning literature [8]. Decision Trees are a model of structuring the data in a hierarchy based on various characteristics in order to make some decisions. The combination of decision trees in the RF method works both for making predictions and categorizing data, and the results are promising when it comes to predicting financial market movements [9]. A combination model, combining Support Vector Machine (SVM) and k-Nearest Neighbors (kNN) similar to that by various researcher. Nevertheless, model considered variable weighting, so that compared to the performed hyper models reported without variable weighting, [10], model show superior performance metrics. Researchers developed a model utilizing an Artificial Neural Network (ANN) to predict the trend of a fund linked to the S&P 500 index. The model combined several forecasting elements: foreign exchange rates, worldwide stock indices, US Treasury yields together with returns from big capital businesses. The findings rather revealed that this strategy did not lead to much better predictive accuracy [11]. Another study [12] found researchers decomposing four

market index time series data to their elements. Next, they predicted closing prices and peak values using a multidimensional k-Nearest Nearest Neighbor (kNN) method. It surpassed results of alternative techniques such as ARIMA (Autoregressive Integrated Moving Average) and regular kNN.

There is so much literature that has been dedicated to technical indicators and employing them as signals for finding the best times to purchase or sell stocks [13]. In a paper [14], the authors added news indicators obtainable via Google News, Wikipedia page views, and information regarding the company products. These supplementary indicators were mixed with conventional signals obtainable via technical analysis (TA) to predict the price movement of the subsequent day. Using methods including Artificial Neural Networks (ANNs), Support Vector Machines (SVMs), and decision trees, the method shown prediction efficiencies of more than 80%. The method shown in the integrated framework [15] utilized Bayesian probability theory and mixed it with neural networks (NNs) and made predictions that exceeded the results obtained through SVMs and conventional NNs. Mo and Wang [16] introduced neural networks customized with time-related functions. Their main goal was to predict the relationships and interconnections between the stock market indices in China and the United States. The methodology they developed shows potential for enhancing the optimization of asset portfolios. Pei et al. [17] altered conventional neural networks by including Legendre 2 polynomials within the networks' internal layers. Moreover, they introduced a particular temporal function in updating the connection weights between these layers. Their method was distinct as they concentrated on forecasting moving averages (MA) of various time spans for prices, instead of directly predicting prices or their directional movement. The authors proposed that this method helps reduce the influence of incidental factors when determining the trend's direction.

Recent research also emphasizes the usefulness of hybrid and the representation-learning methods in financial forecasting. As an example, Kashif and Slepaczuk [18] reveal that LSTM with ARIMA is more robust because both linear and nonlinear components are jointly modeled and that Bieganski and Slepaczuk [19] indicate that supervised representation learning with autoencoders has a higher predictive power because the features are extracted with targeted optimization. These results support the increasing relevance of the integrated modeling approach in the professional field of finance. Recent research [20, 21] by authors indicates that the measures of the standard regression might not be the best representation of the trading objectives and that direction sensitive loss functions might be a better measurement of the predictive optimization as being more aligned with economic results. The search of such objective functions is a significant addition to the current framework. In a study [22] it is stressed that the stability of models across regimes, the interpretability of features and rigid time-series validation procedures are necessary to make financial forecasting reliable. All of this underlines the need to test predictive models in a strong and open-minded experimental framework.

2 Methodology

The empirical basis of this study consists of historical price data for the NSE Bank Nifty index obtained from Yahoo Finance, a widely used financial data service, where Yahoo Finance serves as the data provider and the National Stock Exchange (NSE) of India is

the originating exchange. The daily dataset spans a 20-year period from September 17, 2007, to February 7, 2024, and corresponds to official index values published by the NSE. In addition, corresponding 1-min intraday Bank Nifty index data obtained from publicly available Kaggle datasets are used for testing and supplementary analysis. Kaggle data authenticity was assessed by reconstructing daily OHLC candles from 1-min data and comparing them with Yahoo Finance daily records, with close agreement indicating internal consistency. As the Bank Nifty is an index series, no corporate action adjustments procedures are required, and all data reflect standard NSE trading sessions. Although the evaluation period includes multiple market regimes, including periods of elevated volatility, a formal subperiod analysis (e.g., pre/post structural changes or crisis periods) is left for future work and may provide further insight into strategy stability. The study is done for the standard NSE trading hours of 09:15 to 15:30 IST. No positions are held overnight with all trades being closed during the same trading session to avoid being caught in the overnight gaps. In order to have good quality data and model reliability, forward fill and backward fill were used to deal with missing values. The decision between the two was arrived at because of the quality of the data and the requirement of having temporal consistency and logical continuity in the data set. The outliers were not removed but limited for the sake of preserving the realness of the trading situation. The advantage of this method is that it brings the natural variability of the market without extreme values damaging the performance of the model.

A fully overlapping sliding window approach with per-window Min–Max normalization was used with the sample comprising of the observations of the past window-size (WS) observations and the target being the normalized next value. This configuration allows the model to adapt to the local variations in the scale, mitigates the non-stationarity of the time series, maintains short to medium term temporal dependencies and enhances the learning stability by limiting the inputs to a consistent range. This was to eliminate the possibility of features with bigger values dominating others during training which would enable the model to learn more effectively and reliably on all inputs. Most of the literature papers that were reviewed divided the datasets into about 70–80 percent and 30–20 percent training and testing respectively. In exceptional cases, data partition was of three parts, i.e. training, validation and testing [23]. Nevertheless, in this study 70:10:20 is the training, validation and testing ratio. Model development follows a walk-forward validation framework with chronological data splits. Hyperparameters are optimized using training and validation windows only, to prevent information leakage, while the final test period remains untouched until the end of the study to ensure unbiased out-of-sample evaluation. Hyperparameters for XGBOOST, LSTM, and the hybrid models (XGBOOST + LSTM and XGBOOST + MA) were optimized using a hybrid tuning strategy that combines iterative manual selection, as adopted in the present work, with preliminary analysis to determine the batch size, number of epochs, and window size, followed by Bayesian optimization–based refinement using validation performance. The final hyperparameter values correspond to the best-performing model configurations. Manual hyperparameter tuning is guided by three evaluation metrics namely Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and a goodness of fit parameter R^2 . Which are used to finalize model parameters by assessing the agreement between predicted outcomes and actual values. In the case of the XGBOOST + MA proposed framework, the technical indicator based on the

closing price series were used as feature engineering. Precisely, window length moving averages of 10, 20, and 50 periods were calculated and incorporated as more predictive variables. These moving-average (MA) elements represent short, medium and long-term market trends and hence, these elements improve the power of the model to learn temporal dependencies as well as, they smooth the high-frequency noise that exists in raw price data. This improved set of features makes the model more accurate in its prediction and more stable in its learning behavior especially when the market is experiencing volatile behavior.

Daily trading strategy is adopted to determine the profits and losses using the model predictions. The predictive models were developed using daily market data, where features and targets were constructed at a daily frequency to generate next-day trading signals. To evaluate the economic feasibility of the generated signals under realistic market conditions, the execution layer of the framework was implemented using 1-min intraday data. The intraday data were used exclusively for trade execution simulation, including entry/exit price determination, stop-loss evaluation, and transaction-level performance measurement. No intraday variables were used during model training, validation, or signal generation. This separation guarantees the consistency between the learning objective and the frequency of data used for prediction, while enabling the use of higher-resolution data to enhance the realism of backtesting by taking intraday volatility and the impact of execution into account. Consequently, the framework is a multi-resolution evaluation approach where the predictive signals on a daily basis are evaluated under intraday execution constraints without violating information leakage or distributional inconsistency. The prediction problem is set up as a one-step-ahead regression problem, where the goal is to predict the next-bar price of the Bank Nifty index given a sliding window of historical prices. Although the models provide prices continuously, trading decisions are based on the predicted direction of the price. A buy signal is issued when the predicted value of the price at time $t + 1$ is greater than the value of the price at time t . Otherwise, a sell signal is issued. In the strategy, risk management has also been implemented such as the use of stop-loss, position sizing, to reduce the possible losses and ensure that exposure is kept under control. Risk management is one of the pillars of a successful trading plans, which is supposed to focus on reducing the chances of making a loss and maximizing the chances of profitable trading. The ability to have a clear view of risk exposure will allow the traders to retain their capital which will allow them to remain in the market. Stop-loss mechanisms are applied to automate the determination of exit, which excludes fear or greed in trading. On the same note, smaller position sizes enable the traders to spread their capital in more than one trades, hence not vulnerable to the success of a single trade hence less overall risk. Transaction charges refer to the expenses that are involved in carrying out trades in financial markets, and play an important role in determining the profitability of a trading strategy. The current model takes the form of the transaction fee of 2 per cent on each transaction calculated based on the realized gain or loss. This formulation does not represent an exchange-level microstructure model, where costs are typically proportional to traded notional, and therefore the reported results should be interpreted within the context of this simplifying assumption. Future work may extend to more detailed modeling of accounts for transaction charges, latency and slippage effects. However, its consideration makes the performance metrics of the strategy to be able to capture the real-world trading conditions. The inclusion of

transaction charges allows traders to make better decisions, improve their plans and to control general cost effectively, which encourages sustainable and realistic trading activities. In the case of this study, a starting trading capital of 100,000 (Rs 100,000) is used. Combining machine learning and financial analysis, the proposed research is expected to produce actionable information about trading performance and focus on rigorous evaluation and visualization methods to promote the increased transparency and make the better-informed decisions.

The compound effect may generate a significant amount of trading outcome in the long-term with proper risk management. Through the power of compounding, trading methods can become strong tools of long-term wealth creation, and the importance of disciplined implementation and highly risk management cannot be underestimated. Regularity in returns, which is supplemented by measures like stop-loss mechanisms and size of positions, is a requisite in leveraging compounding in a long-term sustainable manner. The paper has shown that a trading strategy with relatively low performance, as measured by the naked eye, can gain substantial amounts of capital over a period by applying automated risk management and the concept of compounding.

2.1 Framework setup

The hardware configuration, including CPU/GPU and memory specifications used for model training and evaluation, is provided in Table 1 “Computer System Configuration”.

Figure 1 represents the traditional methodology [6, 10, 24–26] that has been traditionally used in stock market prediction studies, which utilize AI/ML. Further, it introduces the suggested extended framework that should be used to understand the proposed framework in a more comprehensive analysis, which guarantees a clear and certain choice of the model (M1/M2/M3/M4.../Mn) used to do real trading and make profits in the stock market. The given framework is not merely dedicated to attaining great levels of prediction accuracy to be better than comparative models. Rather it focuses more on a pragmatic approach by incorporating a fairly efficient model with compounding, risk management, and realistic profit calculation to guarantee a more indispensable and sustainable trading strategy framework. A model can perform averagely when using the metric visualizations and generate good financial results when applied to the proposed framework. Such visualization of average performance might be attributed to the generalization capacity of the model which makes it more robust in the real-world trading. Conversely, a high-performance model may be prone to overfitting thus less flexible to changes in the market in the long run. The presented framework has presented several options of which model to use in the trade. Among the important methods of searching and developing the framework are:

1. Developing and testing various trading strategies to determine the most effective model.

Table 1 Computer system configuration

Processor	Intel(R) Xeon(R) Gold 6244 CPU @ 3.60 GHz, 3592 MHz, 8 Core(s), 16 Logical Processor(s)
RAM	256 GB
System type	Windows 11 Pro for Workstations, x 64-based processor
Programming language	Python 3.11.7 64-bit
Additional tools	Spyder IDE 6.0.2

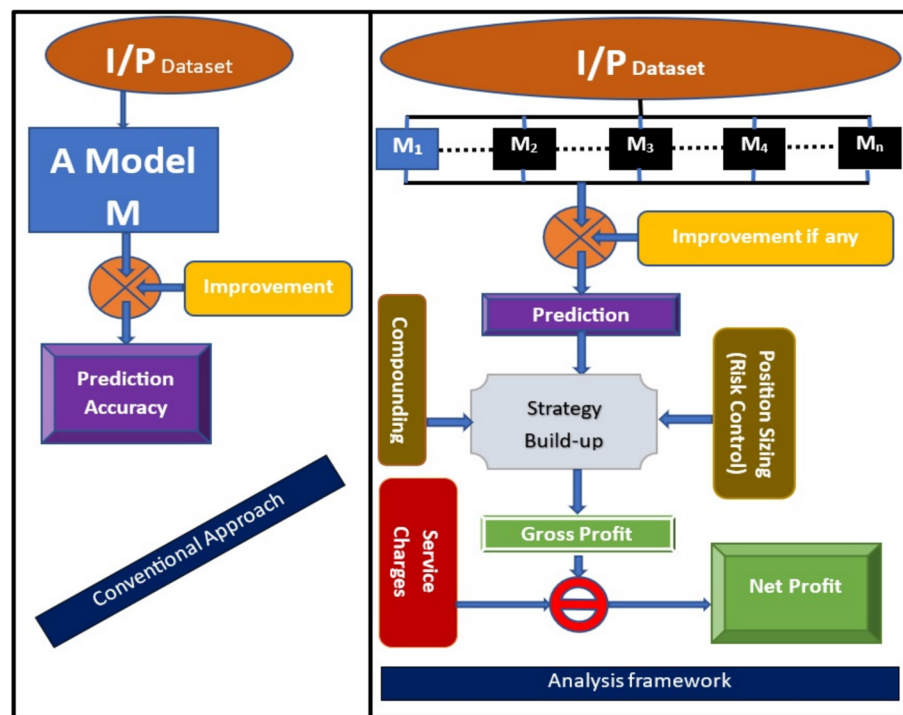


Fig. 1 Proposed framework versus conventional approach of majorities of research papers based on AI-ML Stock Market analysis

2. Analyzing outcomes across different compounding frequencies, such as daily, weekly, or monthly, to optimize profit potential.
3. Experimenting with different risk exposures (position sizing) to identify the best balance between risk and return.
4. Evaluating drawdown levels and optimizing the model based on an individual's risk tolerance and capacity to manage drawdowns.

The four models assessed in this paper are the common predictive methods of LSTM [27–31] and XGBOOST [24, 32–34] that are eminent in the current stock market studies. All programs of improvement are equally spread to all the comparative models so that there can be fair evaluation. All the models are exposed to the same trading strategy with the same settings in terms of compounding, position sizing as well as cost of transacting. The profit growth that follows is then discussed and other significant trading metrics such as drawdown, the number of profit days at different levels, number of winning days and other analytical parameters are also considered.

2.2 Algorithmic setup

Figure 2 does illustrate the advanced algorithmic setting configuration which provides a vivid picture of the working flow which is adopted in this study. Hyperparameters of machine learning models such as LSTM networks and XGBOOST are very sensitive to window size (WS), batch size (BS), a number of epochs (E). The size of a window determines on the history data size on which to perform prediction, with the benefits large windows in LSTM have the ability to learn long-range dependencies, whereas large windows are susceptible to overfitting; whereas small windows in LSTM may miss important trends. XGBOOST, it influences feature extraction and learning through time.

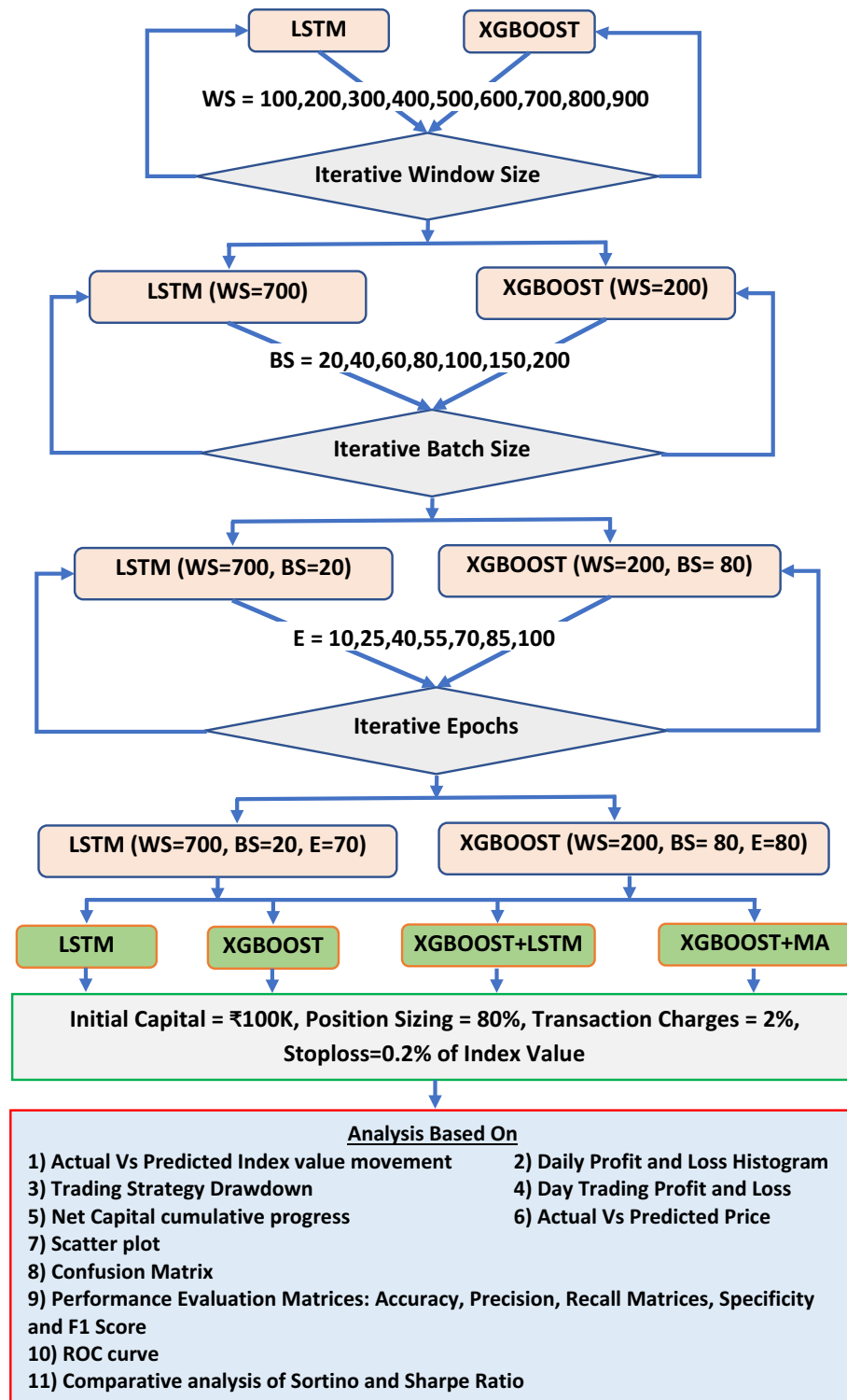


Fig. 2 Data flow diagram for complete analysis

Training dynamics are characterised in terms of batch size, small batches allow accurate weight updates, but are subject to noisy gradients, whereas large batches make gradients more stable and hardware efficient, but might converge to poor local optima.

The epochs are a central factor in determining the extent of data in which a model learns. Train too little and you run the risk of underfitting—where the model makes little more than a surface getting at the real patterns of interest. Go too far and (particularly with models such as LSTM, that learn in sequence) you will fall into the overfitting trap with the model starting to memorize instead of generalise. XGBOOST is more likely to deal with this more gracefully, frequently stopping early to halt training once validation performance starts showing a downturn, trying to prevent unnecessary computation and overfitting. It is not to find the right balance by turning the knobs here and there but a strategic process. Good hyperparameter tuning is therefore the art of finding the right balance between bias and variance in the sense that a model does not merely do well on things it has been given, but that it is reliable that it will also transfer to conditions that it has never been exposed to in the market before.

In order to truly understand the differences between LSTM and XGBOOST, this paper will dig into the comparison between the two models on a massive number of hyperparameter combinations. It discusses the performance of the models of both when they are tested using different window sizes (WS: 100 to 900), batch sizes (BS: 20 to 200) and epochs (10 to 100). As an alternative to using these tweaks as a trial set, it is done in steps, starting with a fixed batch size of 55 and 25 epochs to first stabilize on the best window size. When that is determined, the next step is to tweak the batch size, and then a final step of further-tuning is the number of training epochs. In the process, the performance is evaluated with four important measures including MAE, MSE, RMSE and R2 which together informs us not only of the proximity of the predictions, but also how predictable the model is in general. The structured, though flexible, approach taken by the study puts the guesswork behind, and builds a strong case of which of these configurations really work in practice, not only in the sense of crude prediction performance, but also practical utility as a trading strategy. Algorithm 1 Framework Algorithm.

```

Func Data_Preparation() {
    Load      dataset;
    Apply      sliding-window-technique;
    Normalize  each window(Min-Max-scaling);
    Split      dataset(training,validation,testing,70-10-20); }

Func Performance_Metrics() {
    Evaluate   MAE;
    Evaluate   MSE;
    Evaluate   RMSE;
    Visualize(actual, predicted); }

Func Trading_Simulation() {
    Iteratively For each prediction:
        Generate      Signals(buy/sell);
        Capital =      capital * (Position_Size%/100);
        PnL =          Trade(Signal,capital,SL%, Tran_Charges);
        Cumulative_PnL = Cumulative_PnL + PnL;
        Datalog        History; }

Main Start:
    Input=> (Historical data (Close prices) of the Bank Nifty index);
    Input=> (Param: WS,E,BS,Initial_Capital,Tran_Charges,SL%,Position_Size%);
    Call    Data_Preparation;
    Build   Model;
    Train   Model;
    Prediction   Model;
    Normalized(predictions_values,actual_values);
    Call      Performance_Metrics();
    Call      Trading_Simulation();
    Compute   drawdown;
    Calculate  risk-adjusted metrics;
    Evaluate   classification metrics;
    Output=> (Predicted prices, performance metrics (MSE, RMSE, MAE, Sharpe
    Ratio, Sortino Ratio, etc.), trading profit/loss, and visualization outputs);

End;

```

Algorithm 1 Framework algorithm

This research first compares two simple models: LSTM with window size 700, batch size 20 and 55 epochs and XGBOOST with window size 200, batch size 80 and 25 boosting rounds. In order to enhance the predictive power a hybrid model (XGBOOST+LSTM) was conceptualized, integrating the best features of XGBOOST and LSTM in order to take advantage of the benefit of gradient boosting and sequential learning. Since technical indicators are the central theme in traditional stock trading, a hybrid model, XGBOOST+MA, was created that combines moving average (MA) as the central feature. The model XGBOOST+MA model was superior to the standalone models and the hybrid XGBOOST+LSTM. An exhaustive comparison between LSTM, XGBOOST, XGBOOST+LSTM and XGBOOST+MA established the fact that the XGBOOST+MA model was the best throughout, providing the maximum returns and thus establishing itself as the most effective technique to be used for the financial dataset in question. The long-term functioning of the framework is described in Algorithm 1.

3 Discussion and results

The present study evaluates performance relative to model-based baselines under a consistent trading and risk management framework. Comparison with simple or no-edge strategies such as random signals, moving-average crossover rules, or buy-and-hold approaches is left for future investigation and would provide additional context for interpreting absolute performance levels. To ensure a fair comparison, all studied models are trained using features derived from the same historical price data. Differences in input representation reflect the modeling paradigm rather than access to additional information. In addition, the present study focuses on a single index, and further validation across multiple assets and extended out-of-sample periods is required to assess generalizability. To prevent the risk of mixing up predictive performance and trading profitability, all models were considered in a common trading and risk management context. This experimental design enables the effect of the predictive model to be evaluated independently of execution rules and also emphasizes the strong influence of risk management and portfolio design beyond prediction accuracy alone. Trading decisions are made from the direction model predictions. A long position is opened under the influence of a buy signal, and a short position is opened under the influence of a sell signal. All entries and exits are made at the end of the respective 1 min candle after signal generation. Positions are exited either as soon as the predefined stop-loss level is reached or at the end of the trading day's session automatically thereby ensuring that no positions are carried over the night. To keep the frequency of trading under control and avoid over trading effect, the strategy permits to make maximum one trade per trading day. No fixed level of take profit is imposed; the profits are taken from signal-driven exits and risk-controlled stop-loss execution. Position sizing is based on a fixed allocation strategy where 80% of available capital is allocated per trade without using leverage and margin, and the remaining capital is used as a buffer for transaction costs and short-term changes in the market. The performance indicated in the reported portfolio growth is the performance under the particular experimental set-up and should not be taken as a guaranteed or universally achievable outcome without a strategy execution framework. In [35], the authors assess predictive performance using a hybrid forecasting based on prediction features such as differences in closing prices, EMA, and moving average. The scope of their study is limited to prediction accuracy which does not take into consideration practical trading elements such as position sizing, stop loss strategies or capital compounding.

3.1 Parameter tuning

This section describes the step-wise exploration of the entire framework. The first aim is to identify the best WS for LSTM and XGBOOST models with the BS and number of E as constants at 55 and 25, respectively. The test dataset's actual and predicted values are compared on the basis of the metrics MAE, MSE, RMSE, and R^2 . Figure 3 is a comparative graph for predicted and actual BANK Nifty index values corresponding to different values of WS from 100 to 900 at steps of 100. Besides, Fig. 4a, b and c offers an overall comparative assessment of MAE, MSE, and RMSE for the XGBOOST and LSTM models. The output obviously demonstrates the fact that models produce their minimal error measures based on different values of WS. To be more precise, the minimum MAE, MSE, and RMSE is obtained using XGBOOST with a value of WS of 200 and by LSTM for a value of WS of 700. Figure 4d and e are a combined plot of all three

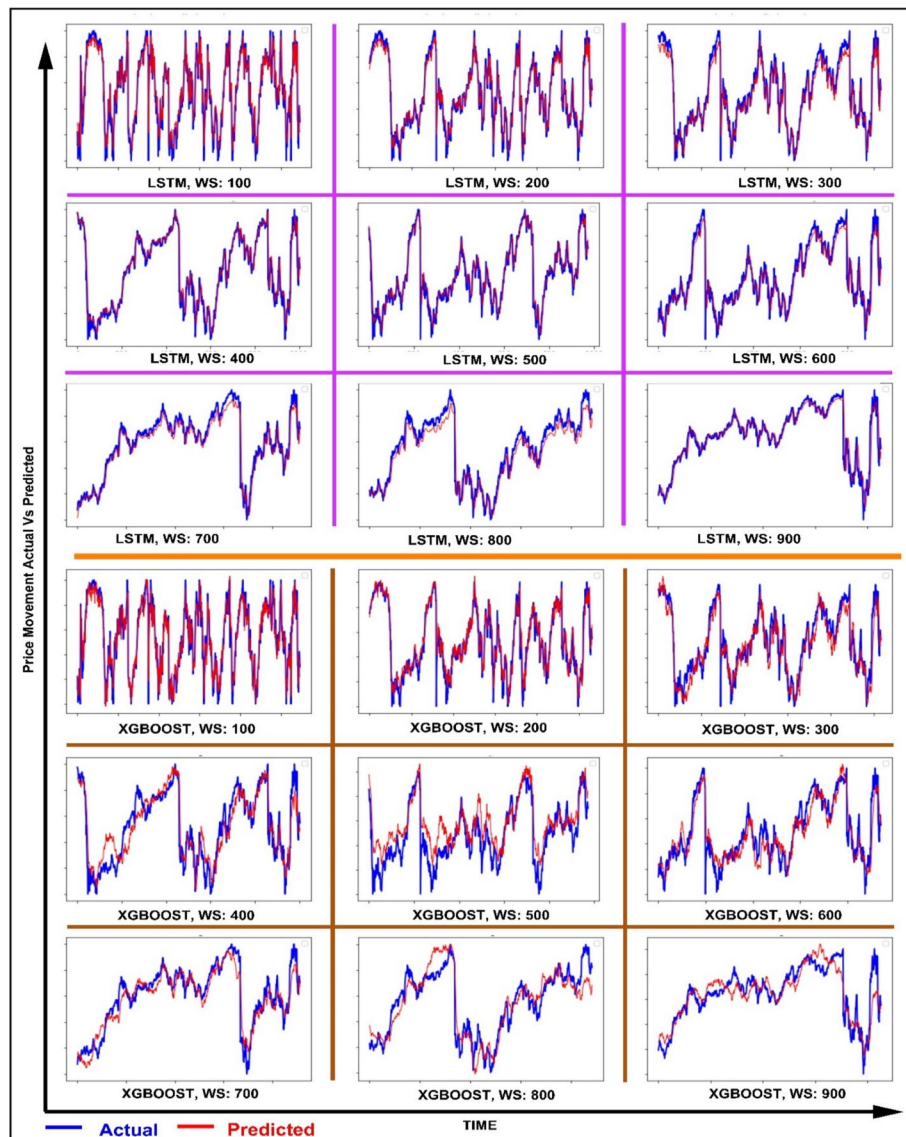


Fig. 3 LSTM and XGBOOST Models Price Movement for NSE Bank Nifty Index using different window size at default batch size = 55 and epochs = 25

measures of error of LSTM and XGBOOST. These plots affirm the fact that the minimum values of error occur at WS 700 for LSTM and WS 200 for XGBOOST. Preliminary analysis reveals that the LSTM model exhibits favorable performance with both window sizes of 200 and 400, suggesting their potential suitability. A more in-depth evaluation will be conducted in subsequent stages of this study, after determining the optimal epoch and batch size parameters. Lastly, Fig. 4f presents a graphical representation of the quality of model fit between XGBOOST and LSTM at WS values of 200 and 700 for both, thus affirming the improved performance of these settings for each model, respectively. Figure 5 depicts a batch size (BS) study in the form of a comparison of forecasted and actual price plots once window size (WS) is set as 200 and the number of epochs (E) is set at 25. The study shows the effect of different BS on model performance. Figure 6a and b depicts that, for WS = 200 and E = 25, BS = 80 provides the lowest error measures in Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean

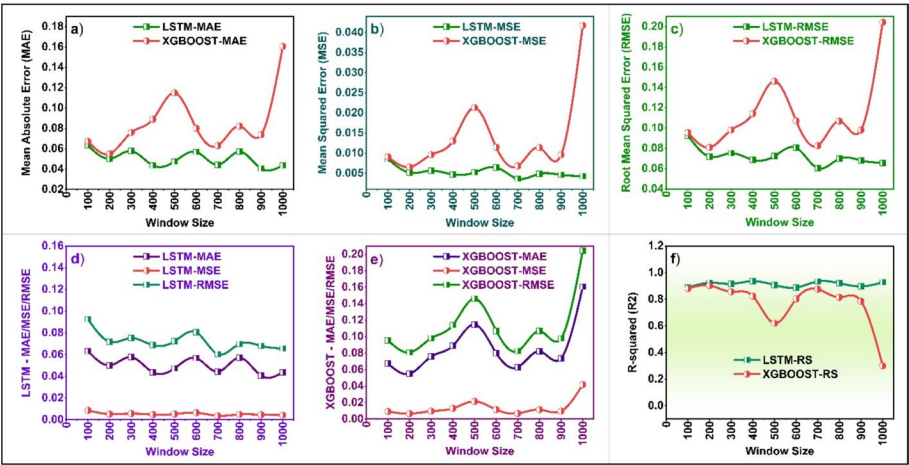


Fig. 4 Comparative Analysis of LSTM vs XGBOOST on MAE, MSE, RMSE and RSquare with respect to different window size at default batch size=55 and epochs=25

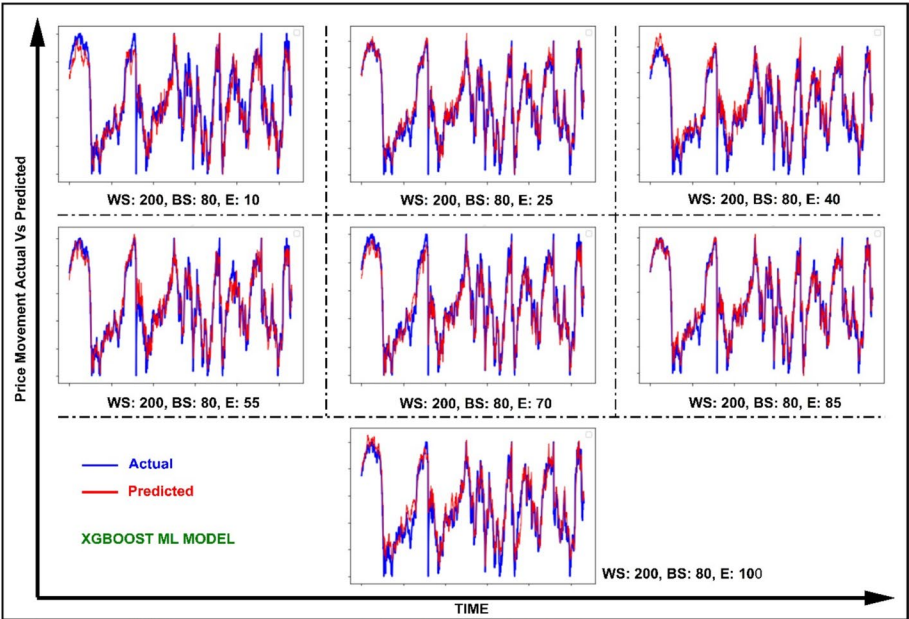


Fig. 5 XGBOOST Model Price Movement for NSE Bank Nifty Index using batch size=80 and window size=200 at different epochs

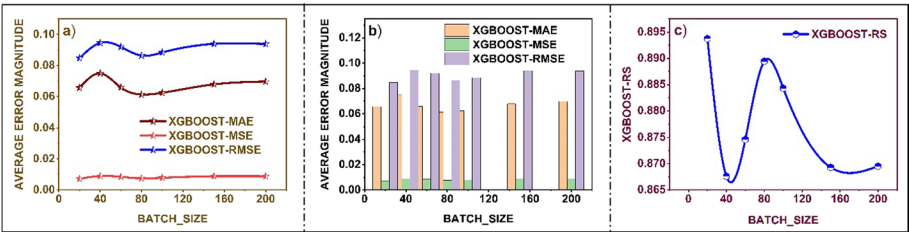


Fig. 6 Analysis of XGBOOST on MAE, MSE, RMSE and RSquare with respect to epochs=25 and window size=200 at different batch size

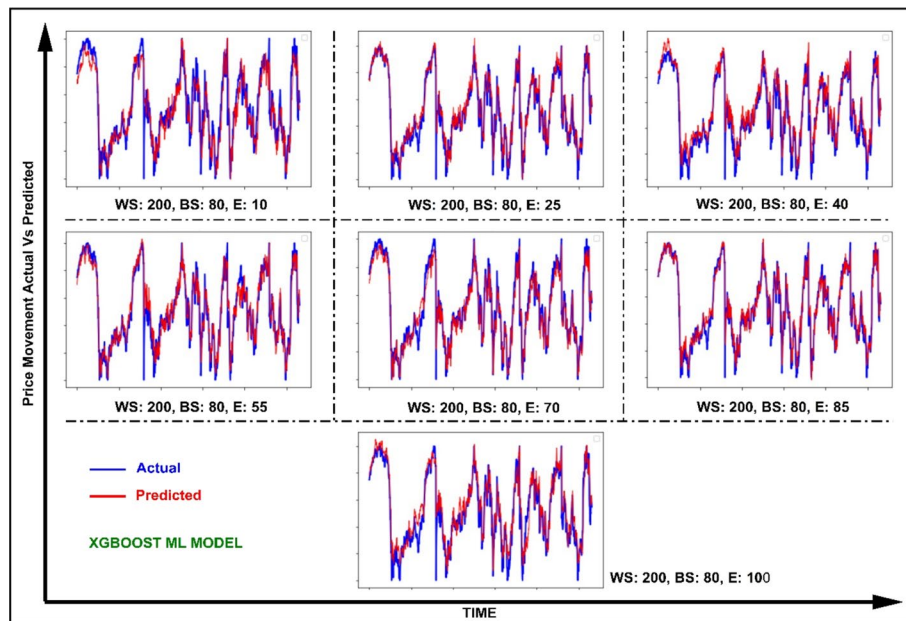


Fig. 7 XGBOOST model price movement for nse bank nifty index using batch size=80 and window size=200 at different epochs

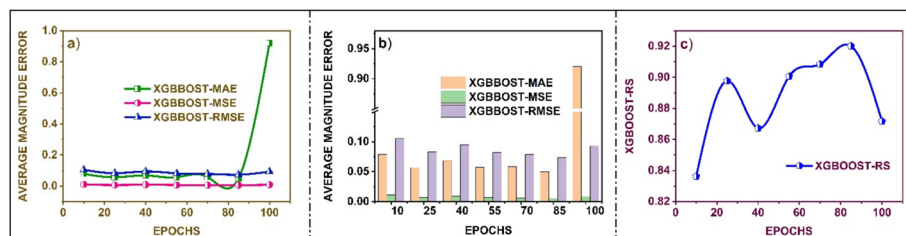


Fig. 8 Analysis of XGBOOST on MAE, MSE, RMSE and RSquare with respect to batch size=80 and window size=200 at different epochs

Squared Error (RMSE). Additionally, in Fig. 6c, $R^2=0.894$ confirms the superior performance of XGBoost at BS = 80. The R^2 measure, close to 90%, confirms a good fit to the data, thus confirming the robustness of this setting. Using a fixed WS of 200 and a BS of 80, the optimal epoch was identified by examining the alignment between the actual and predicted price patterns, as illustrated in Figs. 7 and 8. From Fig. 8b, it is evident that the error magnitude reaches its minimum at epoch 25. Additionally, Fig. 8c demonstrates that the model achieves a good 90% level of data fitting at the same epoch. These observations confirm that epoch 25 provides the most reliable learning state for the proposed model under the specified configuration. Similar to XGBOOST, iterative method was used to adjust the LSTM model parameters with a view to attaining the minimum error between the predicted and actual prices. Experiment trials were run for a WS of 700, BS iteratively changed, and E iteratively tried up to 100. Figure 9 encapsulates the iterative process of identifying the best batch size in determining the best selection. The predicted vs. actual price analysis of the test data with E = 25 and WS = 700 is illustrated in Fig. 9. The results, as illustrated in Fig. 10a and b, confirm that minimum prediction error was attained at BS = 20 with the performance metrics being MAE = 0.327, MSE = 0.0023, and RMSE = 0.048. The corresponding R^2 value of 0.96 further ensures

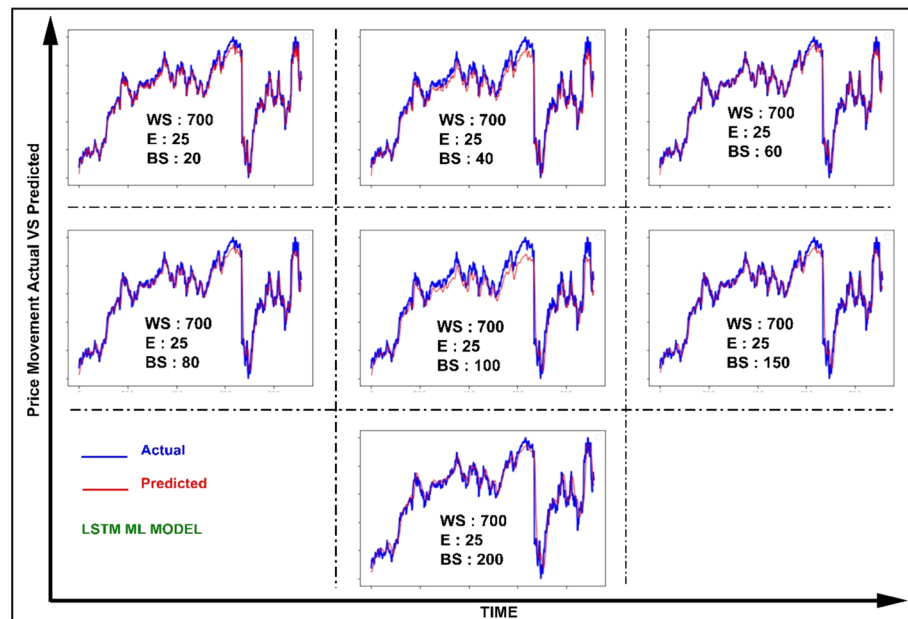


Fig. 9 LSTM model price movement for NSE Bank Nifty Index using epochs = 25 and window size = 700 at different batch size

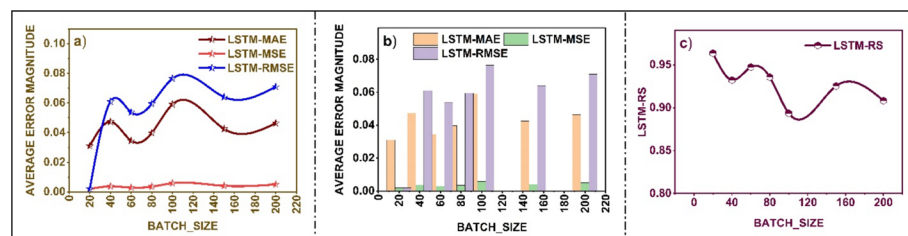


Fig. 10 Analysis of LSTM on MAE, MSE, RMSE and RSquare with respect to epochs = 25 and window size = 700 at different batch size

the excellent performance and good fit of the model to the data. The optimal number of epochs was then established using the predicted vs. actual price graph illustrated in Fig. 11. The final epochs for the LSTM model were established at 70 since this setting gave the minimum error metrics across MAE, MSE, and RMSE while ensuring high predictive accuracy as shown in Fig. 12. The relative comparison of the R^2 parameter reveals the better predictive capability of LSTM model ($R^2 = 0.98 +$, Fig. 12c) than that of XGBOOST model ($R^2 = 0.92 +$, Fig. 8c). Although high R^2 values were observed, all preprocessing, feature construction, and tuning procedures were designed to be strictly causal to avoid look-ahead bias, and predictive accuracy alone is not treated as evidence of trading robustness. It means that the LSTM model fits the real data better, representing the hidden patterns and being more accurate in the process of modeling the intricate dynamics of the data. Nonetheless, high R^2 value indicates that a model has high correlation between the predicted and actual figures which means that it can capture complex trends in the training dataset. This closer fit however can cause overfitting, which is a phenomenon in which the model is over-fitting the training set and thus it can inadequately predict new data. In real life trading situations, this non-generalization can lead to erratic results because the mode might be wrong in its interpretation of new market

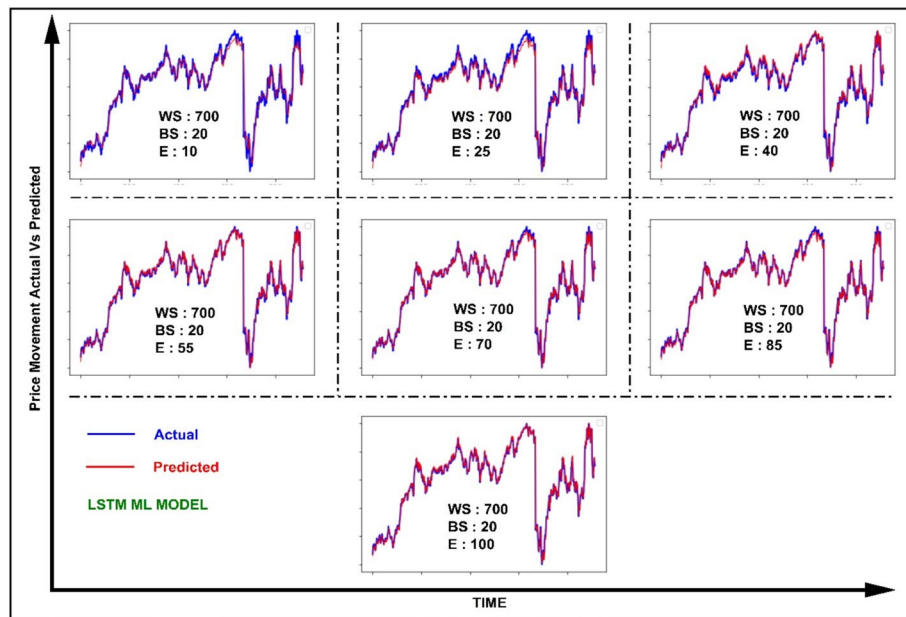


Fig. 11 LSTM model price movement for NSE Bank Nifty Index using batch size = 20 and window size = 700 at different Epochs

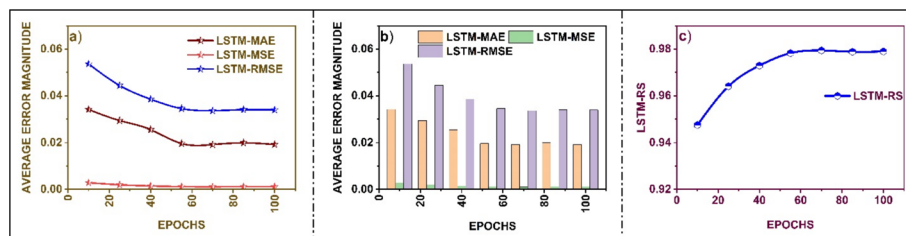


Fig. 12 Analysis of LSTM on MAE, MSE, RMSE and RSquare with respect to batch size = 20 and window Size = 700 at different epochs

trends, or overreact to insignificant changes, or even fail to cope with changing trends. To counteract these problems, extensive validation using out-of-sample data sets, regularization techniques, and extensive testing under various market scenarios are essential to ensure reliability and adaptability in real-world applications. The central theme of this paper is not to investigate the possible limitations of overfitting or the challenge of generalization in real-world trading conditions. However, these features offer a significant opportunity for future research and can be investigated further from this angle. Subsequent sections of this paper reveal that a loose-fitting model is better than a tight-fitting model. This finding reveals the practical benefits of models with enhanced generalization ability since they are more able to cope with evolving market conditions and new information in real-world trading conditions. As noted earlier, the final LSTM configuration, employing a batch size of 20 and 70 epochs with a window size of 700, was evaluated against alternative window sizes of 200 and 400. This comparison was conducted to reconfirm the optimal configuration by identifying the setting that minimized the deviation between actual and predicted values. As illustrated in Fig. 13a–c, the configuration with WS = 700, BS = 20, and E = 70 produced the lowest MAE, MSE, and RMSE values, and was therefore selected as the final and most accurate configuration for this study.

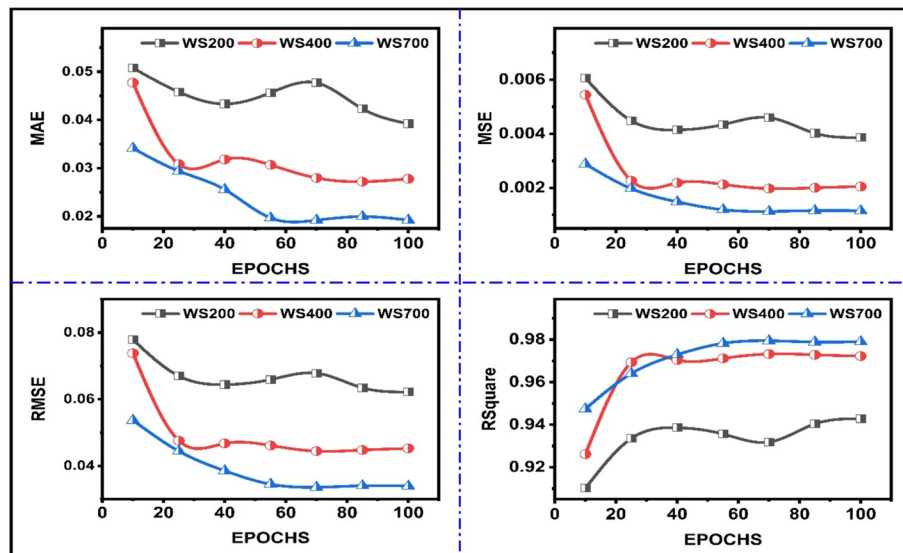


Fig. 13 Comparative analysis of LSTM model for different window sizes at different Epochs with batch size = 20

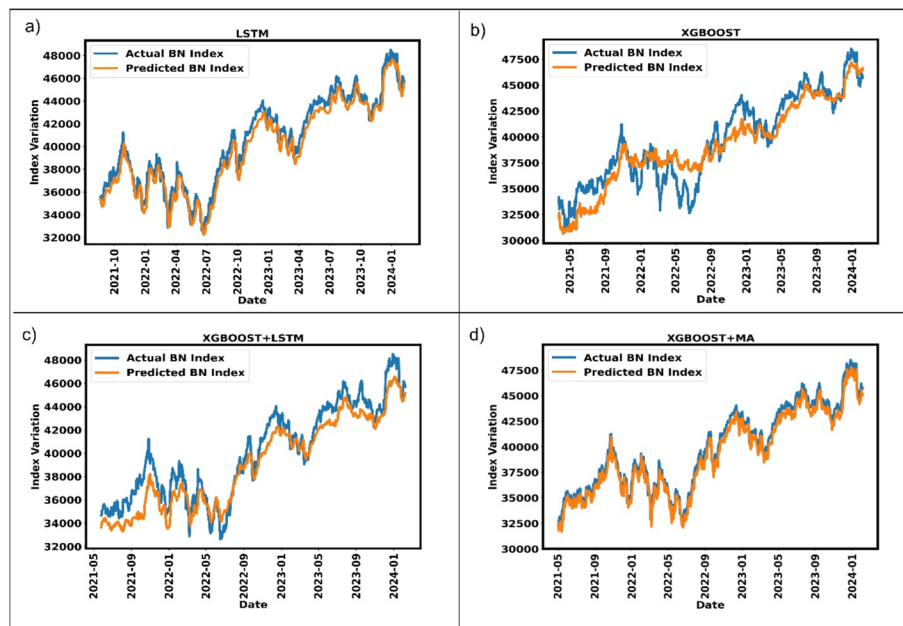


Fig. 14 Actual versus predicted Index value movement of all four models

3.2 Comparative analysis

Here, we conduct a comparative study of four different models, including LSTM, XGBOOST, XGBOOST + LSTM, and the last proposed model, XGBOOST + MA, using different performance indicators. A set of comparative figures have been progressively discussed and appraised to formally prove the benefits of the proposed model against the rest of the three options. Figure 14 shows that the LSTM model is better than the XGBOOST and XGBOOST + LSTM models in the ability to understand the underlying dynamics of time-series data. Seemingly the LSTM model is efficient, which is shown by the close match between the actual and predicted index values, but is computationally expensive and can also be overfit, particularly when used on smaller datasets. Overfitting

problem is reduced by XGBOOST because of its regularization. In the present case of Bank Nifty, the computing time of the LSTM model under the same hardware condition took 315.61 s- far much longer than both XGBOOST and XGBOOST + MA models, which took 3.6 s and 1.39 s, respectively. The high time of computations of 366.81 s was posted by the hybrid XGBOOST+LSTM model which is mainly associated with LSTM element. The quick reactivity of data is essential towards effective risk management in the live trading environment. Therefore, models that require a relatively shorter time of computation are usually preferred to those ones whose computational demands are high. The chart of the XGBOOST + MA model has different features as compared to the other three prediction charts in the Fig. 14. The resultant difference can be attributed to the fact that the moving average technical indicator has been included into the data thus its influence on the index prediction results has been substantial. The XGBOOST + MA model is the end result of a combination of the low computational call rate of the XGBOOST model at a higher level of predictability as indicated by the chart (Fig. 14d). It should however be noted that a significant correlation between actual and predicted values may not be adequate to guarantee best trading profitability. In this paper, this claim is supported by a detailed discussion of profit and loss forecasting in the following parts. The models are measured based on two main measures, as shown in the histogram in Fig. 15; the total number of instances of profit that has been more than Rs 10K, and the total number of instances of losses under the same framework structure. XGBOOST + MA model has significantly more instances of profit exceeding Rs 10,000, 14 instances, compared to the 1, 3, and 1 recorded by the LSTM, XGBOOST and XGBOOST + LSTM models respectively. The number of corresponding losses of the strategies founded on the LSTM, XGBOOST, XGBOOST + LSTM, and XGBOOST + MA models are 342, 348, 340, and 287, respectively. The fact that the loss rate has reduced and that the number of profit cases over the 10 K mark are higher

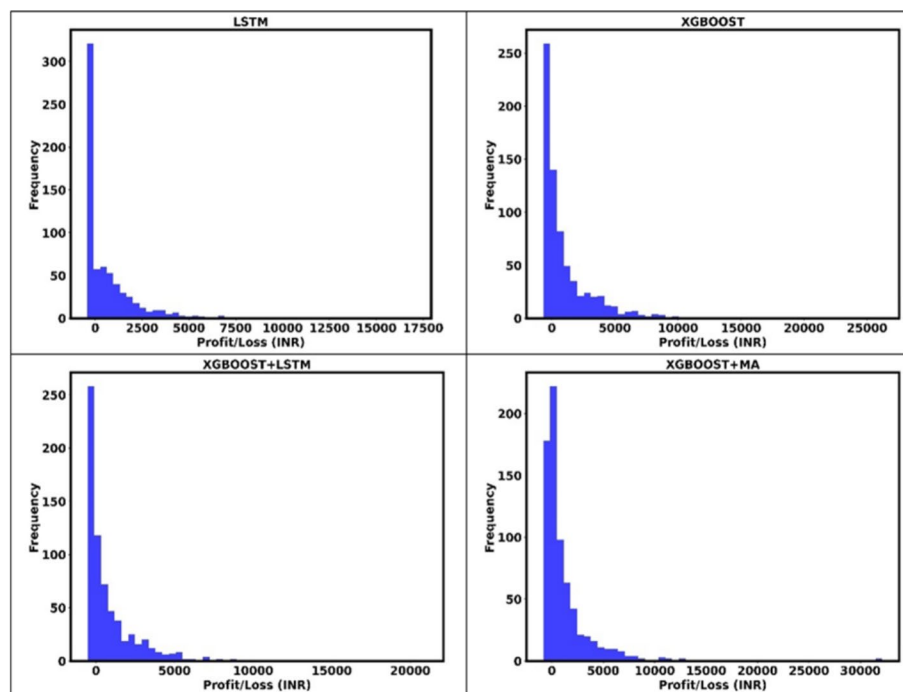


Fig. 15 Daily profit and loss histogram for all four models

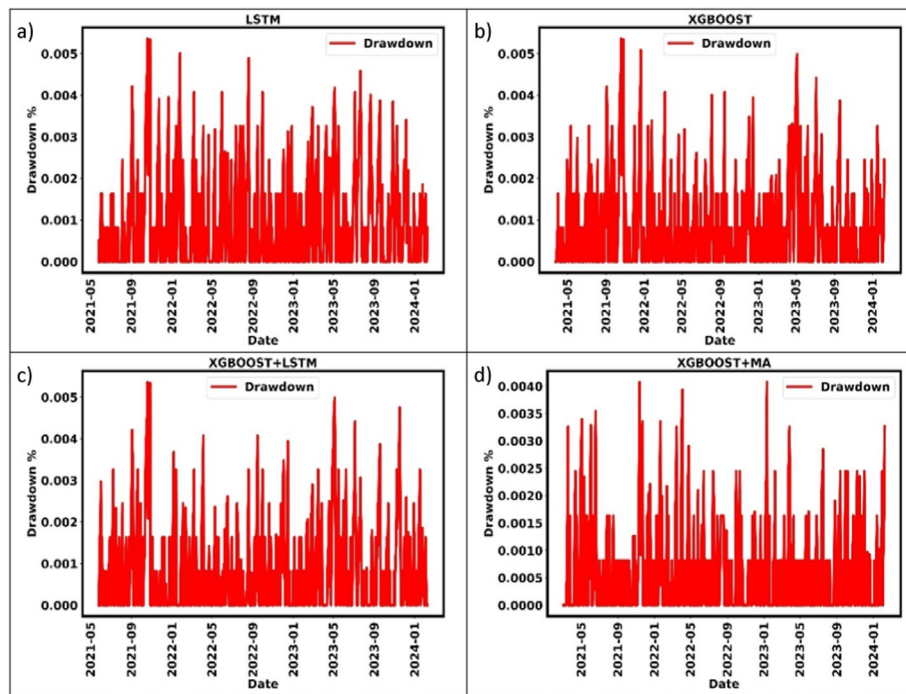


Fig. 16 Trading strategy drawdown of all four models

Table 2 Drawdown (DD) insights of all four models

	LSTM	XGBOOST	XGBOOST + LSTM	XGBOOST + MA
DD Days %	56.28	55.22	56.73	47.45
NO DD days count	292	317	289	351
DD > 0.001% count	215	207	214	147
DD > 0.002% count	122	101	109	55
DD > 0.003% count	54	44	47	15
DD > 0.004% count	17	15	18	2
DD > 0.005% count	3	3	2	0

indicates that the trading strategy developed under the proposed XGBOOST + MA approach should be associated with better prospects of steady profit growth over time.

The drawdown analysis in Fig. 16 indicates that the XGBOOST + MA model experiences the minor maximum drawdown of all the analyzed ones, which means that XGBOOST + MA is better at risk-management and it is more efficient at preserving its capital in the conditions of market volatility. The XGBOOST + MA approach has the lowest period of drawdown, which is recorded as 47.45%. Conversely, the LSTM, XGBOOST, and XGBOOST + LSTM models have drawdown days of 56.28, 55.22 and 56.73 percentage respectively. Moreover, it exceeds those of its counterparts in the number of drawdown days on the drawdown percentages of 0.001, 0.002, 0.003, 0.004 and 0.005 as summarized in Table 2. Profit and loss analysis is one of the essential points of the analysis of any financial approach, which directly demonstrates the feasibility and functionality of the approach in practice.

The daily trade profit and loss (PnL) of all four models are presented in Fig. 17 and the detailed descriptive analysis of the PnL is presented in Table 3.

According to the PnL performance, the strategies are ranked as shown below: XGBOOST + MA, XGBOOST, XGBOOST + LSTM and LSTM. XGBOOST + MA model

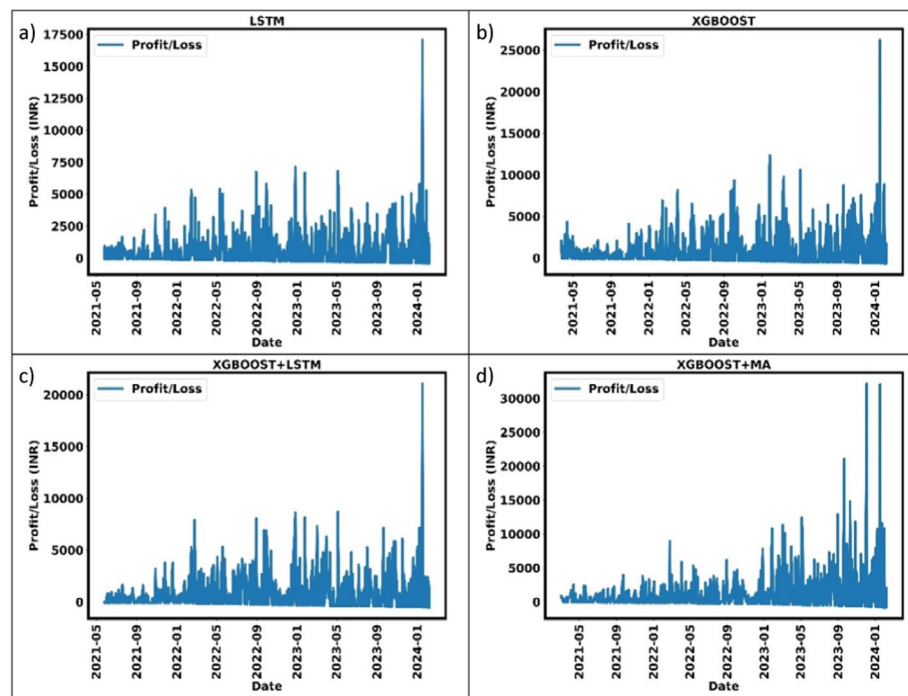


Fig. 17 Day trading profit and loss of all four models

Table 3 Profitable Trade Count of all four models

Total count of trades with below features	LSTM	XGBOOST	XGBOOST + LSTM	XGBOOST + MA
Profit > 0	327	361	329	403
Profit < 0	342	348	340	287
Profit > ₹25,000	0	1	0	2
Profit > ₹20,000	0	1	0	3
Profit > ₹15,000	1	1	1	4
Profit > ₹10,000	1	3	1	14
Profit > ₹5000	14	43	18	52
Profit > ₹2500	66	126	80	120

has the most number of profitable trades of 403, which is higher than the other models of 361, 329, and 327 respectively. In like manner, XGBOOST + MA model documents the least loss-making trades, which amounts to 287 compared to 348, 340 and 342 of the other models, respectively.

Moreover, the XGBOOST + MA optimistic performs better than the other models in as far as the number of daily occurrences of profits of more than the threshold of Rs 25K, Rs 20K, Rs 15K, Rs 10K, Rs 5K and Rs 2.5K. This is mainly because the XGBOOST + MA model is more skillful in identifying and modelling market trend dynamics. The lag effect of moving averages is likely to make a traditional moving average (MA)-based model ineffective in a sideways market or highly volatile markets. But the hybrid XGBOOST + MA model can eliminate this weakness. There are also LSTM and XGBOOST models which exhibit varying strengths based on market conditions. LSTM works best in trending markets and XGBOOST is flexible in irregular environments. In essence the LSTM + XGBOOST hybrid model is usually better in performance as it is able to make the most of both methods. Although both models have different advantages and disadvantages, when judged against a dispassionate 20-year set of data,

which is characterized by various market trends, it can be seen that XGBOOST + MA model is much more robust and has better performance in terms of generating capital gains, as Fig. 18 demonstrates. Initially with an investment of Rs 100,000, the investments increase to Rs 562,182 with LSTM, Rs 822,373 with XGBOOST, Rs 602,361 with the LSTM + XGBOOST hybrid, and Rs 1,002,803 with the XGBOOST + MA model respectively. The respective capital growth of about 5.6 times, 8.2 times, 6 times and maximum performance of 10 times was witnessed in XGBOOST + MA model. The reported portfolio growth should be interpreted within the context of the adopted trading assumptions, including position sizing and compounding effects. The present study does not claim the existence of a persistent predictive edge, and comparison with no-edge baselines such as randomized or rule-based strategies remains an important area for future investigation.

Figure 19, Scatter plots provide a simple way of examining the performance of stock prediction models. Such plots are a good visual representation of the degree to which the predicted values are in line with the actual data, the patterns, possible biases, and outliers can be identified. The high concentration of points along the diagonal line indicates that the prediction accuracy is high with few errors. Whereas the distribution of a large spread or deviation from the diagonal points indicates that there are areas of prediction error. Figure 19 depicts that four plots have distinct patterns of dispersion: XGBOOST + MA model has a wide distribution of data points with little overlapping or proximity. The wide dispersion is an indication of poor fit to data, low predictive accuracy, and the highest level of error of all the models used. It however has the highest profitability, which is attributed to the good use of risk management strategies and the compounding principle in the system of trading. The adoption of an optimum stoploss mechanism protects the trading mechanism against the negative effect of outlier.

The confusion matrices presented in Fig. 20 provide a comprehensive evaluation of model performance by detailing the counts of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). These values, as depicted in the figure, form the basis for systematically analyzing and comparing the effectiveness of various stock prediction models. The metrics derived from the confusion matrix are defined using the following formulas:

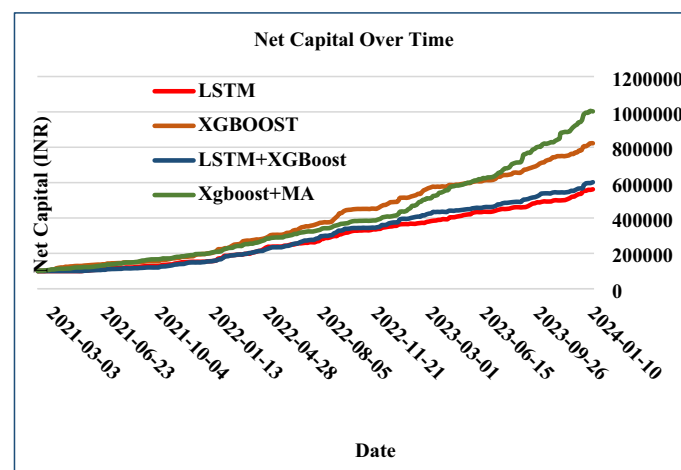


Fig. 18 Net capital cumulative progress of all four models

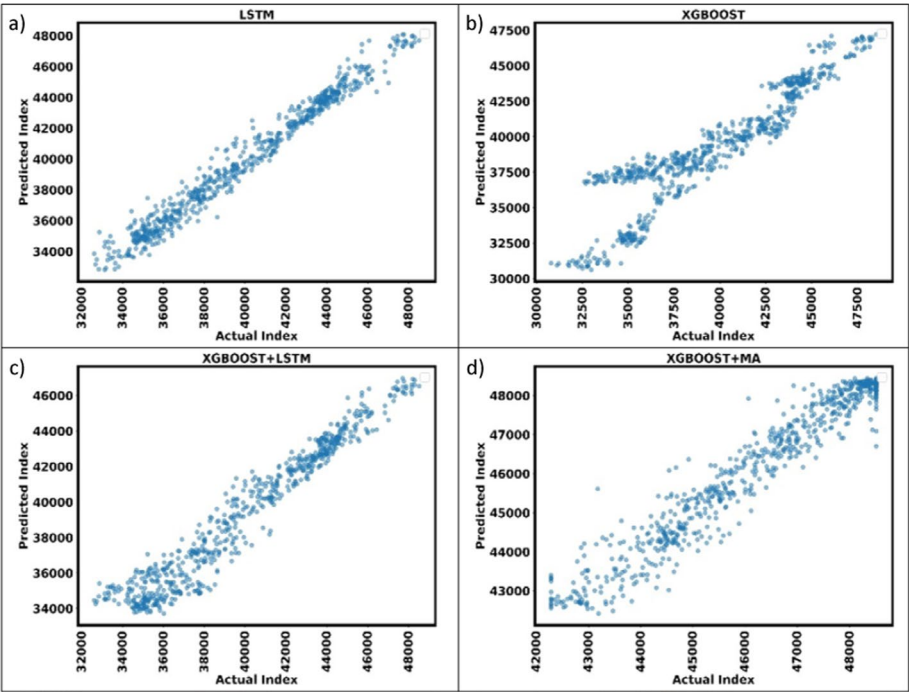


Fig. 19 Actual versus predicted price scatter plot of all four models

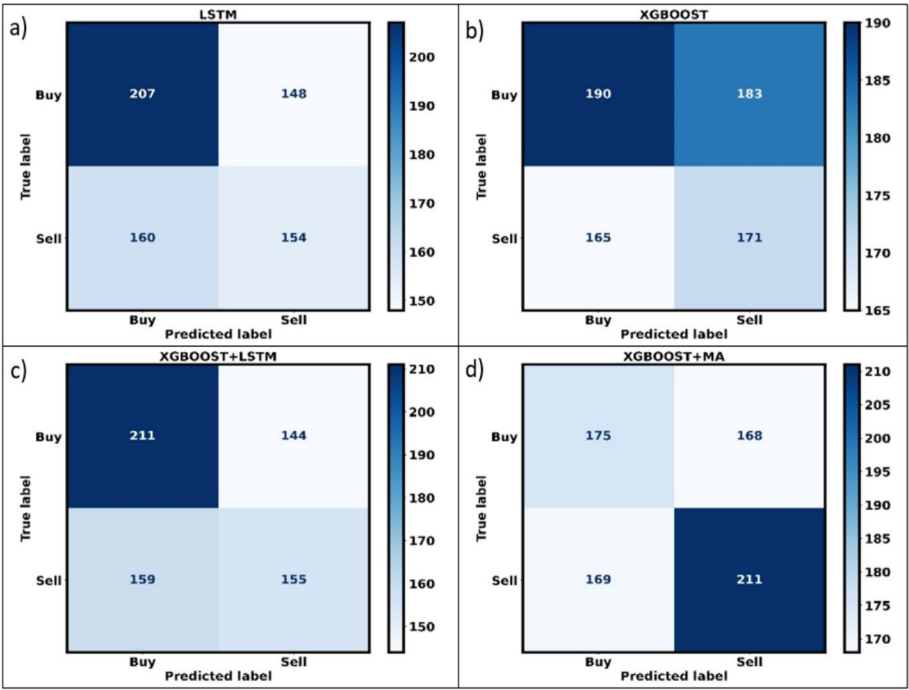


Fig. 20 Confusion matrix of all four models

- Accuracy: Measures the overall correctness of the model [36].

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$$

- Precision: Focuses on the reliability of positive predictions [36, 37].

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- Recall (Sensitivity or True Positive Rate—TPR): Measures how well the model identifies true positives [36].

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- Specificity (True Negative Rate—TNR): Measures how well the model identifies true negatives [38].

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

- F1 Score: Balances precision and recall for imbalanced datasets [36].

$$\text{F1Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

To assess the performance of each model by examining the results for accuracy, recall, precision, specificity, and F1 score. Choose the metric that aligns most closely with oneself trading objectives: accuracy reflects overall performance, recall helps minimize missed opportunities (false negatives), precision reduces incorrect positive predictions (false positives), and F1 score provides a balanced assessment of precision and recall. Select the model that demonstrates the strongest performance in metrics that best align with oneself trading priorities, whether it is minimizing losses or maximizing gains. The evaluation of higher and lower values of metrics helps determine how well a model meets the specific goals and constraints of the stock prediction application: a) Focus on recall for opportunity maximization. b) Prioritize precision and specificity for risk-averse strategies. c) Use F1 score for balanced decision-making in dynamic or volatile markets. Selecting the right model requires aligning its performance with the application's primary objectives and constraints. High Specificity ensure that only reliable and accurate predictions are acted upon.

According to Fig. 21, XGBOOST+MA model with Specificity value of 0.56 shows a conservative strategy, avoiding unnecessary risks and protect capital. This was further validated by analyzing the capital growth performance illustrated in Fig. 18. The Receiver Operating Characteristic (ROC) curve is a graphical plot of a model's ability to distinguish between different classes at different levels of thresholds. The curve is a very important tool employed in the assessment and comparison of the classification ability of stock forecasting models. Models with higher AUC values and whose curves plot nearer to the top-left corner of the graph are more reliable and in agreement with prognostication objectives. The ROC curves in Fig. 22 characterize the connection between the true

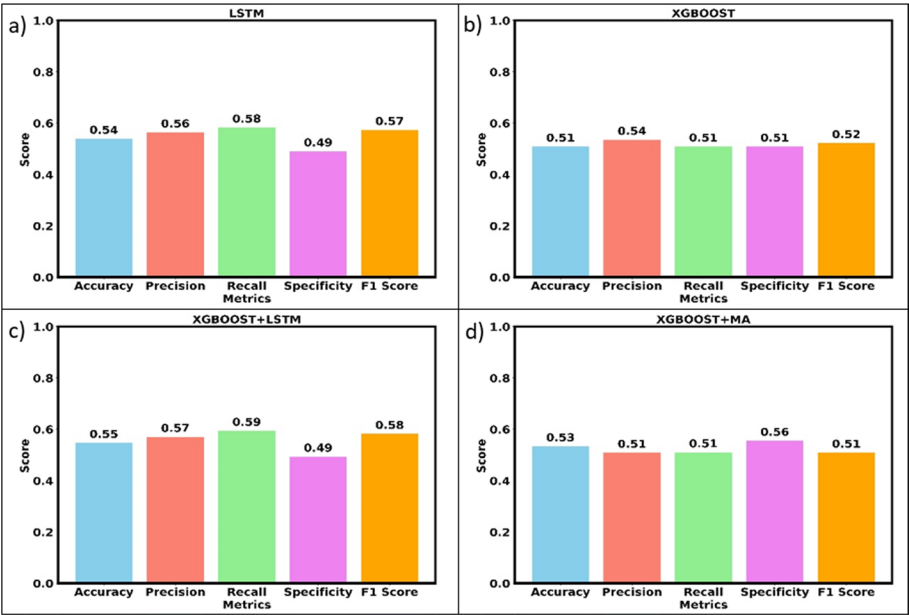


Fig. 21 Performance evaluation matrices all four models

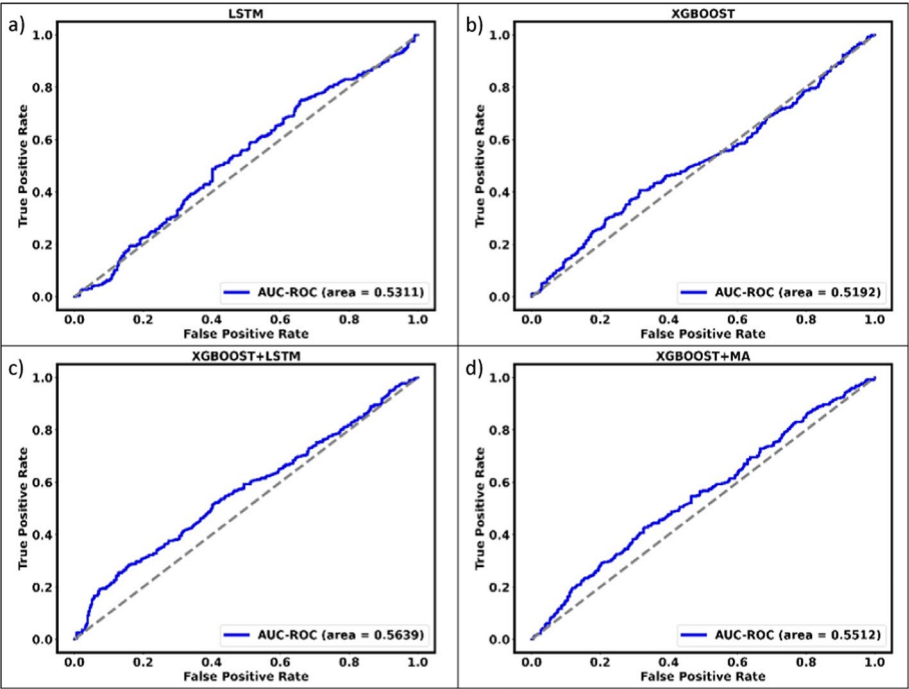


Fig. 22 ROC curve of all four models

positive rate (TPR) and the false positive rate (FPR) for a series of threshold values and therefore effectively characterize the tradeoff between sensitivity and specificity. Sensitivity measures a model’s performance in identifying positive instances correctly, while specificity measures its performance in identifying negative instances correctly. Among the models analyzed, the XGBOOST + LSTM combination showed the highest reliability with an AUC of 0.5624. The XGBOOST + MA model followed closely with an AUC of 0.5512. In contrast, the individual LSTM and XGBOOST models recorded lower AUC

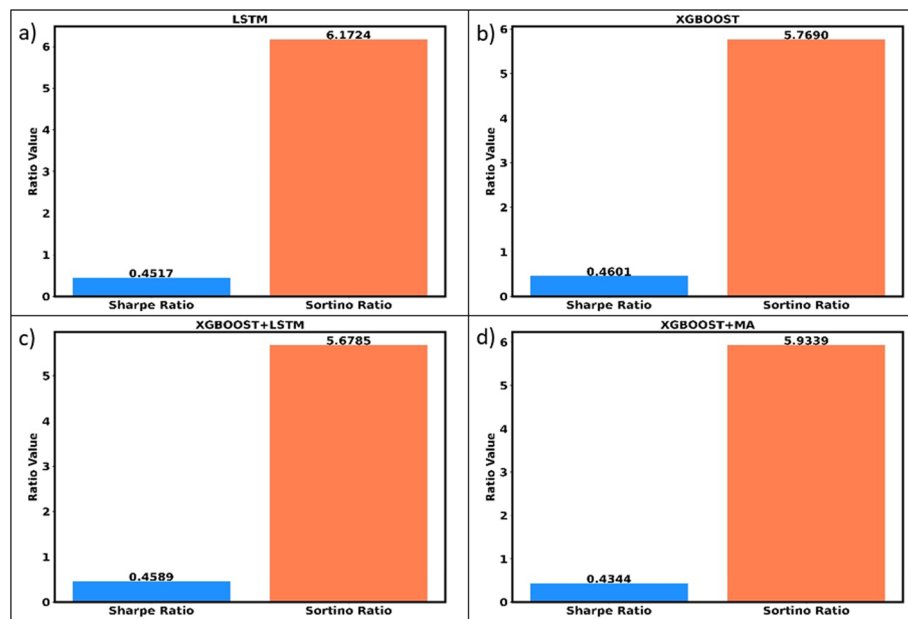


Fig. 23 Comparative analysis of Sortino and Sharpe ratio of all four models

values of 0.5311 and 0.5192, respectively, highlighting their standalone performance. To prioritize minimizing losses, a model with a higher Sortino Ratio, albeit a lower Sharpe Ratio, is usually preferred. The Sortino Ratio is especially helpful in assessing strategies or models that aim to minimize losses in volatile market conditions. Comparing both ratios provides one with better understanding of the effectiveness and risk-adjusted performance of a stock forecasting model. Figure 23 presents a comparative study of the Sortino and Sharpe ratios of all four models considered analyzed, the XGBOOST + LSTM combination showed the highest reliability with an AUC of 0.5624. The XGBOOST + MA model followed closely with an AUC of 0.5512. In contrast, the individual LSTM and XGBOOST models recorded lower AUC values of 0.5311 and 0.5192, respectively, highlighting their standalone performance. To prioritize minimizing losses, a model with a higher Sortino Ratio, albeit a lower Sharpe Ratio, is usually preferred. The Sortino Ratio is especially helpful in assessing strategies or models that aim to minimize losses in volatile market conditions. Comparing both ratios provides one with better understanding of the effectiveness and risk-adjusted performance of a stock forecasting model. Figure 23 presents a comparative study of the Sortino and Sharpe ratios of all four models considered. The results indicate that, among the four models considered, the proposed XGBOOST + MA model does not achieve the highest Sharpe ratio (0.4344), it demonstrates competitive downside-risk performance as reflected by the Sortino ratio (5.9339), indicating the influence of risk-control mechanisms on realized trading outcomes. These comparisons provide empirical insight into risk-adjusted behavior rather than statistical superiority, as formal significance testing is beyond the scope of the present study. Hence, the trading strategy with XGBOOST + MA is extremely preferred among the other models considered, as it is a risk-efficient trading model.

4 Conclusion

This paper shows that a simple XGBOOST+MA model that has a rather low predictive performance of 0.53 can produce outstanding financial results when incorporated into a properly designed and developed trading system. The framework that used XGBOOST+MA was the best performing of the four machine learning models that were analyzed in the study- LSTM, XGBOOST, XGBOOST+LSTM and XGBOOST+MA. This practice will transform an average model-based structure into a one that provides outstanding outcomes by combining risk management and compounding strategies. Emotional bias is overcome by automation of trading tactics, which includes position sizing, stop-loss, and trading costs, and allows traders to react to market changes and continue to trade in the market in the long term. Intraday trading is also an additional protection against the risk of being exposed overnight. The XGBOOST+MA model registered a stunning capital growth in sense that the original investment of Rs 100K was increased to RS1002K- a growth of about ten folds. Comparatively, LSTM, XGBOOST, and XGBOOST+LSTM models were able to provide the growth of 5.6X, 8.2X, and 6X, respectively. Moreover, XGBOOST+MA model had the minimum drawdown since the drawdown days only constituted 47.45 percent of the total time, compared to 56.28 percent, 55.22 percent and 56.73 percent of the LSTM, XGBOOST and XGBOOST+LSTM models respectively. The better performance of the XGBOOST+MA model is further emphasized by the fact that it has a higher number of significant profit cases, with 14 incidences of profit of over 10 K, as opposed to 1, 3 and 1 incidences of LSTM, XGBOOST, and XGBOOST+LSTM model respectively. Likewise, the XGBOOST+MA model had the lowest loss instances (287), which is highly better compared to LSTM (342), XGBOOST (348) and XGBOOST+LSTM (340) models in reducing negative occurrences.

The results indicate that risk management and capital compounding play a significant role in determining realized trading performance within the examined framework. These observations, however, should be interpreted in the context of the specific experimental setting adopted in this study and should not be considered universally applicable. Although the outcomes are encouraging, further investigation may focus on hybrid modeling approaches that incorporate additional technical indicators alongside XGBoost to improve predictive capability and to better understand model behavior within the proposed framework. The approach may also be extended to a broader set of stocks and market indices across different exchanges, which could provide deeper insights into its practical applicability. It should be noted that the current framework does not account for execution delays or slippage associated with real trading environments, and therefore backtesting results may differ from live trading performance. Future developments should emphasize responsiveness to real-time market dynamics while incorporating safeguards to minimize the risk of overfitting.

Author contributions

Conceptualization, L.K.S., M.S., A.J., P.S., A.K.G., and A.K.; Formal analysis, L.K.S., M.S., A.J., and P.S.; Investigation, A.J., A.K.G., and A.K.; Methodology, L.K.S., M.S., A.J., and P.S.; Validation, L.K.S., M.S., A.J., A.K., and P.S.; Writing-original draft, L.K.S., M.S., A.J., P.S., A.K.G., and A.K.; Writing-review & editing L.K.S., M.S., A.J., P.S., A.K.G., and A.K.; Supervision, A.J., A.K.G., and A.K.

Funding

Open access funding provided by Manipal Academy of Higher Education, Manipal. No funding is received for this work.

Data availability

All the data are available within the manuscript. For more information, please contact the corresponding authors.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 6 January 2026 / Accepted: 17 April 2026

Published online: 11 May 2026

References

1. Ren T, Li S, Zhang S. Stock market extreme risk prediction based on machine learning: evidence from the American market. *N Am J Econ Finance*. 2024;74:102241.
2. Kim M. Adaptive trading system integrating machine learning and back-testing: Korean bond market case. *Expert Syst Appl*. 2021;176:114767.
3. Kehinde TO, Chan FTS, Chung SH. Scientometric review and analysis of recent approaches to stock market forecasting: two decades survey. *Expert Syst Appl*. 2023;213:119299.
4. Maqbool J, Aggarwal P, Kaur R, Mittal A, Ganaie IA. Stock prediction by integrating sentiment scores of financial news and MLP-Regressor: a machine learning approach. *Procedia Comput Sci*. 2023;218:1067–78.
5. Khan W, Ghazanfar MA, Azam MA, Karami A, Alyoubi KH, Alfakeeh AS. Stock market prediction using machine learning classifiers and social media, news. *J Ambient Intell Humaniz Comput*. 2022;13(7):3433–56.
6. Zhang X, Zhang Y, Wang S, Yao Y, Fang B, Yu PS. Improving stock market prediction via heterogeneous information fusion. *Knowl-Based Syst*. 2018;143:236–47.
7. Wang C, Chen Y, Zhang S, Zhang Q. Stock market index prediction using deep Transformer model. *Expert Syst Appl*. 2022;208:118128.
8. Barak S, Arjmand A, Ortobelli S. Fusion of multiple diverse predictors in stock market. *Inf Fus*. 2017;36:90–102.
9. Krauss C, Do XA, Huck N. Deep neural networks, gradient-boosted trees, random forests: statistical arbitrage on the S&P 500. *Eur J Oper Res*. 2017;259:689–702.
10. Chen Y, Hao Y. A feature weighted support vector machine and K-nearest neighbor algorithm for stock market indices prediction. *Expert Syst Appl*. 2017;80:340–55.
11. Zhong X, Enke D. Forecasting daily stock market return using dimensionality reduction. *Expert Syst Appl*. 2017;67:126–39.
12. Zhang N, Lin A, Shang P. Multidimensional k-nearest neighbor model based on EEMD for financial time series forecasting. *Physica A*. 2017;477:161–73.
13. Cavalcante RC, Brasileiro RC, Souza VLF, Nobrega JP, Oliveira ALI. Computational intelligence and financial markets: a survey and future directions. *Expert Syst Appl*. 2016;55:194–211.
14. Weng B, Ahmed MA, Megahed FM. Stock market one-day ahead movement prediction using disparate data sources. *Expert Syst Appl*. 2017;79:153–63.
15. Yan D, Zhou Q, Wang J, Zhang N. Bayesian regularisation neural network based on artificial intelligence optimisation. *Int J Prod Res*. 2017;55(8):2266–87.
16. Mo H, Wang J. Return scaling cross-correlation forecasting by stochastic time strength neural network in financial market dynamics. *Soft Comput*. 2018;22(9):3097–109.
17. Pei A, Wang J, Fang W. Predicting agent-based financial time series model on lattice fractal with random Legendre neural network. *Soft Comput*. 2017;21(7):1693–708.
18. Kashif K, Ślepaczuk R. LSTM-ARIMA as a hybrid approach in algorithmic investment strategies. *Knowl-Based Syst*. 2025;320:113563.
19. Bieganski B, Ślepaczuk R. Supervised autoencoder MLP for financial time series forecasting. *J Big Data*. 2025;12(1):207.
20. Stefański F, Ślepaczuk R. Informer in algorithmic investment strategies on high frequency bitcoin data. 2025;1.
21. Michańków J, Sakowski P, Ślepaczuk R. Mean absolute directional loss as a new loss function for machine learning problems in algorithmic investment strategies. *J Comput Sci*. 2024;81:102375.
22. Plachta K, Ślepaczuk R. Machine learning for daily return direction forecasting: a comparative study with explainable AI insights. 2025.
23. Nti IK, Adekoya AF, Weyori BA. A systematic review of fundamental and technical analysis of stock market predictions. *Artif Intell Rev*. 2020;53(4):3007–57.
24. Han Y, Kim J, Enke D. A machine learning trading system for the stock market based on N-period Min-Max labeling using XGBoost. *Expert Syst Appl*. 2023;211:118581.
25. Chandola D, Mehta A, Singh S, Tikkiwal VA, Agrawal H. Forecasting directional movement of stock prices using deep learning. *Ann. Data. Sci*. 2023;10(5):1361–78.
26. Al-Khasawneh MA, Raza A, Khan SUR, Khan Z. Stock market trend prediction using deep learning approach. *Comput Econ*. 2024. <https://doi.org/10.1007/s10614-024-10714-1>.
27. Ricchiuti F, Sperli G. An advisor neural network framework using LSTM-based informative stock analysis. *Expert Syst Appl*. 2025;259:125299.
28. Phuoc T, Anh PTK, Tam PH, Nguyen CV. Applying machine learning algorithms to predict the stock price trend in the stock market – the case of Vietnam. *Humanit Soc Sci Commun*. 2024;11(1):393.
29. Parreño SJ. Chaos theory enhanced LSTM model of the Philippine Stock Exchange Index. *SN Comput Sci*. 2024;5(6):779.
30. Dhokane RM, Agarwal S. A predictive model of the stock market using the LSTM algorithm with a combination of exponential moving average (EMA) and relative strength index (RSI) indicators. *J Inst Eng (India) Ser B*. 2024;105(5):1145–57.

31. Balamohan S, Khanaa V, Sivaraman K. Effective stock market pricing prediction using long short term memory-upgraded model (LSTM-UP) on evolving data sets. *SN Comput Sci.* 2023;4(5):658.
32. Zouaghia Z, Kodja Z, Ben Said L. Predicting the stock market prices using a machine learning-based framework during crisis periods. *Multimed Tools Appl.* 2024.
33. Dezhkam A, Manzuri MT. Forecasting stock market for an efficient portfolio by combining XGBoost and Hilbert–Huang transform. *Eng Appl Artif Intell.* 2023;118:105626.
34. Chen W, Chen B, Cai X. Forecasting China's stock market risk under the background of the Stock Connect programs. *Soft Comput.* 2024;28(3):2483–99.
35. Yan K, et al. Flexible target prediction for quantitative trading in the American stock market: a hybrid framework integrating ensemble models, fusion models and transfer learning. *Entropy.* 2026;28(1):84.
36. Olorunnimbe K, Viktor H. Deep learning in the stock market—a systematic survey of practice, backtesting, and applications. *Artif Intell Rev.* 2023;56(3):2057–109.
37. Kaczmarczyk K, Miałkowska K. Backtesting comparison of machine learning algorithms with different random seed. *Procedia Comput Sci.* 2022;207:1901–10.
38. Chatzis SP, Siakoulis V, Petropoulos A, Stavroulakis E, Vlachogiannakis N. Forecasting stock market crisis events using deep and statistical machine learning techniques. *Expert Syst Appl.* 2018;112:353–71.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.